CONTINUACIÓN DE «TESTING DE INTEGRACIÓN»

Testing de integración II

De 7 tests a una estrategia · Diseño según Plataformas de Juego · 120 min

Ing. Elsa Daniela Ramírez · FI – UNJu · 2026

DIAGNÓSTICO · CLASE ANTERIOR

Dónde quedamos: la suite actual

#	Test (TestSuite.cs)	Flujo que cubre	Tipo de caso
01	AsteroidsMoveDown	El asteroide se desplaza	Positivo
02	GameOverOccursOnAsteroidCollision	Asteroide + nave → Game Over	Positivo
03	NewGameRestartsGame	Reinicio (Game Over simulado a mano)	Positivo
04	GameOverStopsSpawningAndDisablesShip	Game Over detiene spawner y nave	Positivo
05	LaserMovesUp	El láser se desplaza	Positivo
06	LaserDestroysAsteroid	Láser + asteroide → destrucción	Positivo
07	DestroyedAsteroidRaisesScore	Láser + asteroide → puntaje	Positivo

💻 **Ver en el código:** TestSuite.cs 53–149

ℹ Todos son casos positivos. Sin negativos, sin bordes, sin HUD, sin limpieza entre tests.

AL FINALIZAR PODRÁN

Objetivos de la clase

- Diseñar casos de integración positivos, negativos y de borde para un flujo de juego
- Preparar fixtures y datos de prueba que aíslen cada test
- Reconocer las causas de un test frágil (flaky) y corregirlas
- Elegir qué dependencias usar reales y cuáles reemplazar por dobles de prueba
- Interpretar un reporte de cobertura sin sobreestimarlo
- Organizar una mini-suite de integración priorizada



ORGANIZACIÓN

Agenda (120 min)

Bloque	Min	Teoría	Demo	Práctica	Resolución
Apertura y diagnóstico	8	5	3	–	–
1. Positivos, negativos y borde	12	8	4	–	–
Actividad 1 · diseñar casos	17	–	–	12	5
Actividad 2 · casos borde	15	–	–	10	5
2. Fixtures, aislamiento y tests frágiles	13	8	5	–	–
Actividad 3 · analizar un test fallido	15	–	–	10	5
3. Dependencias y dobles de prueba	8	6	2	–	–
4. Cobertura y regresión	8	5	3	–	–
Actividad 4 · mini-suite	19	–	–	14	5
Cierre	5	5	–	–	–
Total	120	37	17	46	20

APERTURA

La suite pasa… ¿y el juego?

- ¿Qué pasa si reinicio la partida con asteroides todavía en pantalla?
- ¿Y si dos láseres golpean el mismo asteroide en el mismo instante?
- ¿Qué pasa si un asteroide choca con otro asteroide?
- El reporte de cobertura dice 73,6 %, y Game.cs tiene 100 %. ¿Qué tan protegidos estamos?

IMAGEN

Captura del Test Runner con los 7 tests en verde junto al resumen del reporte de cobertura regenerado (73,6 % de líneas).

❓ ***¿Cuál de estas preguntas responde hoy la suite? (Respuesta: ninguna)***

6

Positivos, negativos y borde

Ejemplo: una puerta con cerradura

Tipo de caso	Qué pregunta	Puerta	Nave + asteroides
Positivo	¿Hace lo que debe con una entrada válida?	La llave correcta abre	El láser destruye el asteroide y suma 1
Negativo	¿Ignora o rechaza lo que no corresponde?	Otra llave no abre	Un asteroide que choca con otro NO da Game Over
Borde	¿Qué pasa justo en el límite?	La llave a medio girar	Un asteroide en y = −5,0 exacto: ¿se destruye?

- Negativo ≠ test que falla: el test pasa si el sistema ignora correctamente la entrada
- En integración, el borde no es solo un número: también tiempo, cantidad, repetición y orden

Cinco dimensiones del borde en integración

Dimensión	Pregunta	En el juego
Valor / posición	¿Justo en el límite?	Asteroide en y = −4,99 / −5,00 / −5,01
Tiempo	¿Justo antes o después de un intervalo?	Disparo a 0,39 s y a 0,41 s del anterior (cooldown 0,4 s)
Cantidad / simultaneidad	¿0, 1 o 2 eventos a la vez?	Dos láseres contra el mismo asteroide en el mismo paso de física
Repetición	¿Qué pasa si se llama dos veces?	NewGame() dos veces seguidas
Orden	¿Y si los eventos llegan al revés?	Un punto que se suma en el mismo frame del Game Over

💻 **Ver en el código:** Asteroid.cs 43–50 (límite −5) · Ship.cs 82–90 (cooldown 0,4 s) · Laser.cs 46–55 · Game.cs 67–80

ℹ Algunos bordes no tienen un resultado esperado obvio: lo decide el diseño, no quien prueba.

Formato de un caso de integración

Campo	INT-AST-11 · Choque entre asteroides (negativo)
Objetivo	Un choque entre dos asteroides no provoca Game Over
Componentes	Asteroid ×2 · física · Game
Entrada	Dos asteroides superpuestos, lejos de la nave
Precondiciones	Prefab Game instanciado · 1 frame (Start ejecutado) · isGameOver = false
Pasos	1. SpawnAsteroid() ×2 2. Ubicar ambos en (6, 3) 3. Esperar 2 FixedUpdate
Resultado esperado	isGameOver = false · ambos asteroides siguen existiendo · score sin cambios
Bug que detecta	Alguien relaja la condición de Asteroid.cs 54 y cualquier choque termina la partida
¿Por qué integración?	Verifica el contrato por nombre entre Asteroid y Game a través de la física real

PRÁCTICA · EN GRUPOS

Actividad 1 · Diseñar casos nuevos (12 min)

- Flujo A: láser destruye asteroide → puntaje
- Flujo B: asteroide choca la nave → Game Over
- Flujo C: Game Over → nueva partida
- Para cada flujo: un caso POSITIVO nuevo y un caso NEGATIVO
- Usar la ficha de 8 campos · no repetir los tests 01–07



✍ ***Actividad: Entreguen 6 fichas (2 por flujo). Pista: ¿qué NO debería pasar en cada flujo?***

RESOLUCIÓN DOCENTE

Actividad 1 · Resolución

Flujo	Caso	Resultado esperado	Bug que detecta
A +	INT-AST-09 · El HUD muestra el puntaje	2 impactos → score = 2 y texto «Score: 2»	Se borra la actualización del texto (Game.cs 85)
A –	INT-AST-10 · Láser contra objeto ajeno	score = 0 · el objeto sigue intacto · sin excepción	Se quita el filtro GetComponent<Asteroid>() (Laser.cs 48)
B +	(ya cubierto por 02 y 04)	—	—
B –	INT-AST-11 · Choque entre asteroides	isGameOver = false	Cualquier choque dispara Game Over
C +	INT-AST-08 · Reinicio tras Game Over real	Nave viva en (0,0,0) · score 0 · spawner activo	RepairShip o BeginSpawning omitidos
C –	NewGame con partida en curso	Estado inicial, sin doble spawner	Ver INT-AST-15 (repetición)

ℹ HUD: scoreText es privado, pero está dentro del prefab Game (UICanvas/ScoreText): el test puede encontrarlo con GetComponentsInChildren<Text>().

PRÁCTICA · ANÁLISIS SIN CÓDIGO

Actividad 2 · Casos borde (10 min)

Una fila por dimensión, con valores concretos y resultado esperado. Marquen con ⚠ las filas cuyo resultado no pueden decidir ustedes.

Dimensión	Valores a probar	Resultado esperado	⚠
Valor			
Tiempo			
Simultaneidad			
Repetición			
Orden			

✍ Actividad: ¿Quién decide el resultado esperado de una fila marcada con ⚠?

RESOLUCIÓN DOCENTE

Actividad 2 · Resolución

Dimensión	Valores	Resultado esperado	⚠
Valor	Asteroide en y = −4,99 · −5,00 · −5,01	Vivo · vivo · destruido (condición estricta «< −5») → INT-AST-12	
Tiempo	Disparo a 0,39 s y 0,41 s del anterior	No dispara · dispara → INT-AST-16 (hoy no se puede automatizar)	
Simultaneidad	2 láseres → 1 asteroide, mismo paso	score +1 y sin excepción → INT-AST-13 (EJECUTADO: suma 2 y lanza excepción)	
Simultaneidad	1 láser → 2 asteroides superpuestos	¿+1 o +2? Decisión de diseño	⚠
Repetición	NewGame() dos veces seguidas	Misma tasa de aparición → INT-AST-15 (EJECUTADO: 8 asteroides vs. 5)	
Orden	Punto sumado en el frame del Game Over	¿Cuenta o no? Decisión de diseño	⚠

Fixtures y datos de prueba

Ejemplo: un laboratorio

- Fixture: el estado conocido del que parte cada test (SetUp) y la limpieza al terminar (TearDown)
- Como un laboratorio: mesada limpia antes del experimento, limpieza al terminar
- En la suite actual, TearDown destruye solo el prefab Game
- Asteroides y láseres se crean como objetos raíz: sobreviven al test y «ensucian» el siguiente
- Game.instance es estático: puede apuntar a un Game ya destruido hasta que corra el siguiente Start()
- Spawner usa Random sin semilla: cada corrida genera posiciones distintas



Ver en el código: TestSuite.cs 40–51 · Spawner.cs 73–99 (Instantiate sin padre) y 103 (Random) · Game.cs 46 y 50 (instancia estática)

BLOQUE 2 · DEMO

Fixture mejorada (PROPUESTA)

```
[UnitySetUp]
public IEnumerator SetUp()
{
    Random.InitState(12345);        // datos reproducibles
    var prefab = Resources.Load<GameObject>("Prefabs/Game");
    game = Object.Instantiate(prefab).GetComponent<Game>();
    yield return null;              // corre Start(): instance listo
}

[UnityTearDown]
public IEnumerator TearDown()
{
    Object.Destroy(game.gameObject);
    var none = FindObjectsSortMode.None;
    foreach (var a in Object.FindObjectsByType<Asteroid>(none))
        Object.Destroy(a.gameObject);   // lo que el test dejó
    foreach (var l in Object.FindObjectsByType<Laser>(none))
        Object.Destroy(l.gameObject);
    yield return null;              // Destroy termina al final del frame
}
```

Qué cambia

- Semilla fija: datos reproducibles
- Un frame de espera: Start() ya corrió
- Limpieza total, no solo del prefab
- Otro frame: el próximo test empieza limpio

ℹ Código propio de la cátedra. La limpieza (UnityTearDown) ya corre en IntegrationHypothesesTests.cs; la semilla no se probó.

BLOQUE 2 · TEORÍA

Tests frágiles (flaky)

Ejemplo: Martin Fowler · «Eradicating Non-Determinism in Tests»

Causa	En la suite actual	Remedio
Tiempo fijo	WaitForSeconds(0,1) depende de la máquina y de la física	Esperar por condición, con tiempo máximo
Estado compartido	Asteroides de tests anteriores siguen en escena	TearDown que limpia todo
Orden	04 cuenta TODOS los asteroides de la escena	Contar solo lo que creó el test
Azar	Posición X aleatoria del spawner	Random.InitState(semilla)
Física	Las colisiones ocurren en FixedUpdate	yield return new WaitForFixedUpdate()

ℹ Flaky: pasa o falla sin que cambie el código. Destruye la confianza en toda la suite.

BLOQUE 2 · DEMO

Esperar por condición, no por tiempo fijo (PROPUESTA)

```
// Antes: espera fija (TestSuite.cs 134)
yield return new WaitForSeconds(0.1f);

// Después: espera hasta que se cumpla la condición o se agote el tiempo
float timeout = 2f;
while (asteroid != null && timeout > 0f)
{
    timeout -= Time.deltaTime;
    yield return null;
}
Assert.IsTrue(asteroid == null,
    "El asteroide debería destruirse al recibir el láser");
```

Por qué

- Pasa apenas se cumple: más rápido
- Tolera máquinas lentas
- El timeout evita esperas infinitas
- «asteroid == null» usa la comparación de Unity para objetos destruidos

ℹ Código propio (PROPUESTA). El mensaje del Assert explica QUÉ se esperaba: ayuda a diagnosticar.

PRÁCTICA · ANÁLISIS

Actividad 3 · Un test que falla (10 min)

```
[UnityTest]  // INT-AST-14 · IntegrationHypothesesTests.cs
public IEnumerator NewGame_RemovesLeftoverAsteroids()
{
    yield return null;                    // Start() asigna Game.instance
    game.NewGame();
    GameObject leftover = game.GetSpawner().SpawnAsteroid();
    leftover.transform.position = new Vector3(6f, 3f, leftover.transform.position.z);
    Game.GameOver();
    yield return null;

    game.NewGame();                       // el jugador reinicia
    yield return null;

    Assert.IsTrue(leftover == null, "NewGame debería eliminar los asteroides previos");
}

✗ FAIL (batchmode 28/09/2026) — NewGame debería eliminar los asteroides previos
  Expected: True   But was: False
```

✍ ***Actividad: 1) ¿Falla el juego o el test? 2) Causa en el código 3) Impacto para el jugador 4) Ticket 5) ¿Y después del arreglo?***

RESOLUCIÓN DOCENTE

Actividad 3 · Resolución

SpawnAsteroid() crea con Instantiate y nunca pasa por el pool

↓

NewGame() → ClearAsteroids() → asteroids.Dispose()

↓

Dispose() vacía el pool… que no contiene los asteroides en pantalla

↓

El asteroide viejo sobrevive y puede chocar la nave recién reparada

- Falla el juego si el diseño exige limpiar; confirmarlo antes de levantar el ticket
- Ticket: «Reiniciar no elimina asteroides previos: posible Game Over inmediato» · Severidad Media · Prioridad Alta
- Arreglo: usar el pool de forma consistente (Get/Release) o destruir los Asteroid en ClearAsteroids
- Después del arreglo, el test se queda en la suite como test de regresión

💻 **Ver en el código:** Spawner.cs 73–99 (Instantiate) y 115–117 (Dispose) · Game.cs 78 (ClearAsteroids) · Spawner.cs 52–53 (creación del pool)

Dependencias y dobles de prueba

Ejemplo: Martin Fowler · «TestDouble» (taxonomía de Gerard Meszaros)

Doble	Qué hace	Ejemplo simple	En el juego
Dummy	Solo ocupa un lugar	Un DNI de muestra para llenar un formulario	Cubo con collider en INT-AST-10
Stub	Devuelve respuestas fijas	Un reloj que siempre dice 7:00	Entrada que siempre dice «Espacio presionado» (PROPUESTA)
Spy	Registra llamadas para revisarlas después	Un contador de timbres	Contar llamadas a AsteroidDestroyed (PROPUESTA)
Mock	Espera llamadas concretas y falla si no llegan	«El timbre debe sonar una vez»	GameOver llama a StopSpawning exactamente una vez (PROPUESTA)
Fake	Implementación simple que funciona	Una agenda en papel en vez de la app	Récord guardado en memoria en vez de un archivo (PROPUESTA)

ℹ En integración, lo que está en la costura bajo prueba va REAL; se reemplaza lo no determinista o lo que queda fuera de alcance.

Caso: la regla del cooldown vive junto al teclado

Ejemplo: INT-AST-16 (PROPUESTA)

- Regla de juego: como máximo un láser cada 0,4 s aunque se mantenga Espacio
- El chequeo de canShoot está en Update(), junto a Input.GetKey(Space)
- ShootLaser() no respeta la regla: dos llamadas seguidas crean dos láseres
- Sin simular el teclado, la regla no se puede probar
- Opción A: Input System + InputTestFixture (requiere un paquete no confirmado en el proyecto)
- Opción B: separar la lectura de entrada detrás de una interfaz y usar un stub (refactor)



💻 **Ver en el código:** Ship.cs 61–64 (regla + tecla) · Ship.cs 77–80 (ShootLaser sin chequeo) · Ship.cs 82–90 (cooldown)

❓ ***Esperado con Espacio mantenido 1 s: ¿cuántos láseres? (Respuesta: 3, en t = 0; 0,4; 0,8)***

Cobertura: qué mide y qué no

Ejemplo: Reporte regenerado: 7 tests, carpeta limpia, 28/09/2026

- Reporte viejo del repo: 65,5 %, mezclando 3 corridas (15/09 y 18/09) y el código de los tests
- Regenerado: 73,6 % de líneas y 87 % de métodos de GameAssembly · Game.cs 100 % · Ship.cs 43 %
- Game.cs 100 %… pero la línea del HUD (85) se ejecuta sin que ningún test la compruebe
- Nunca ejecutado: entrada del jugador, movimiento lateral, asteroide que sale de pantalla, spawn automático
- Cobertura de ramas: 0 de 0 → no se midió

🖼 IMAGEN

Captura del reporte regenerado: resumen por clase (Game 100 %, Ship 43 %) y la vista línea por línea de Ship.cs con las líneas 61–74 en rojo.

💻 **Ver en el código:** Sin cubrir: Ship.cs 61–74 y 99–115 · Asteroid.cs 48 · Spawner.cs 69 · Laser.cs 42 — Ejecutada sin verificar: Game.cs 85

❓ ***Game.cs tiene 100 % de cobertura: ¿está bien probado? ¿Usarían ese número para decidir si el juego está listo?***

Regresión: del bug al test que lo vigila

Se detecta el defecto (Actividad 3: asteroides que sobreviven al reinicio)

↓

Se escribe el test y se ve en ROJO: prueba que el defecto existe

↓

Se arregla el código y el test pasa a VERDE

↓

El test queda en la suite: si el defecto vuelve, se entera la suite antes que el jugador

ℹ Retesting = volver a probar ESE arreglo. Regresión = volver a probar TODO lo que ya andaba (Unidad II).

PRÁCTICA · EN GRUPOS

Actividad 4 · Diseñar una mini-suite (14 min)

- Tienen los 7 tests actuales + los casos nuevos de la clase
- Máximo 12 tests en la suite de integración
- Categorías: Smoke (cada cambio) · Integración · Borde · Regresión
- Definan la fixture común y una convención de nombres
- Decidan qué tests existentes se quedan, se reescriben, se fusionan o salen



✍ ***Actividad: Entreguen la suite en una tabla: test · categoría · cuándo corre · decisión (queda / reescribir / fusionar / sale).***

RESOLUCIÓN DOCENTE

Actividad 4 · Resolución

Categoría	Tests	Cuándo corre
Smoke	02 Game Over por choque · 06 Láser destruye asteroide · 08 Reinicio tras Game Over	Cada cambio (segundos)
Integración	04 reescrito (limpieza + conteo propio) · 09 HUD (absorbe 07) · 10 Láser vs. objeto ajeno · 11 Asteroide vs. asteroide	Cada commit
Borde	12 Límite y = −5 · 13 Doble impacto · 15 Doble NewGame	Cada commit
Regresión	14 Reinicio limpia asteroides (en rojo hasta el arreglo)	Cada commit
Fuera de esta suite	01 y 05 → movimiento de un solo componente · 03 → reemplazado por 08 · 16 → pendiente del refactor de entrada	—

ℹ Fixture común: semilla fija + limpieza total · Nombres: Flujo_Condición_Resultado · [Category("Smoke")]

Cierre

- Una suite solo de casos positivos deja sin probar lo que el juego NO debe hacer
- El borde en integración también es tiempo, cantidad, repetición y orden
- Sin fixtures que limpien, los tests se contaminan entre sí y se vuelven frágiles
- Un doble de prueba se usa para lo que no está bajo prueba, no para todo
- La cobertura dice qué se ejecutó, no qué se verificó

✍ ***Actividad: Ticket de salida: un caso negativo y un caso borde para un juego propio.***

REFERENCIA

Anexo · Catálogo de casos nuevos

ID	Caso	Tipo	Estado
INT-AST-08	Reinicio completo tras un Game Over real	Positivo · flujo	PROPUESTA
INT-AST-09	El HUD muestra el puntaje	Positivo · regresión	PROPUESTA
INT-AST-10	Láser contra un objeto que no es asteroide	Negativo	PROPUESTA
INT-AST-11	Choque entre asteroides no da Game Over	Negativo	PROPUESTA
INT-AST-12	Asteroide que cruza y = −5	Borde · valor	PROPUESTA
INT-AST-13	Dos láseres contra el mismo asteroide	Borde · simultaneidad	EJECUTADO · FAIL
INT-AST-14	NewGame elimina los asteroides previos	Regresión	EJECUTADO · FAIL
INT-AST-15	NewGame dos veces seguidas	Borde · repetición	EJECUTADO · FAIL
INT-AST-16	Cooldown de disparo con tecla mantenida	Dependencia de entrada	PROPUESTA (refactor)

ℹ PROPUESTA: todavía no existe en el proyecto. EJECUTADO: en Assets/Tests/IntegrationHypothesesTests.cs (batchmode, 28/09/2026).

CONTENIDO RESERVADO

Anexo · Para una clase futura

Técnicas

- Integración con sistemas externos: guardado del récord, tabla online
- Input System + InputTestFixture para simular teclado y táctil
- Tests parametrizados ([TestCase], [ValueSource]) para bordes
- LogAssert para esperar o prohibir mensajes en el log

Proceso

- Ejecución en batchmode y CI (tests en cada push)
- Cobertura de ramas y umbrales mínimos
- Tests de rendimiento (Performance Testing)
- Relación con la plataforma: la misma suite en PC y en móvil